

# Centroid & Centroid Decomposition

2026 ALOHA 2학기 고급반

## Motivation

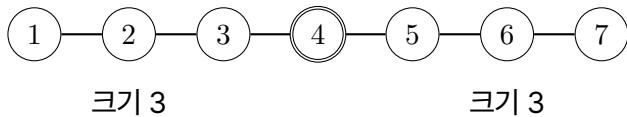
- ✓ 트리에서 모든 정점 쌍을 직접 확인하면  $O(N^2)$  개의 쌍이 생김
- ✓ 예를 들어 길이가 정확히  $K$  인 경로의 개수를 구하고 싶음
- ✓ 정점 하나를 기준으로 경로를 처리하고, 나머지를 작은 문제로 나눌 수 있음
- ✓ 하지만 한쪽으로 치우치게 나누면 시간 복잡도가 줄어들지 않음
- ✓ Centroid는 각 조각의 크기가 절반 이하가 되도록 나누는 기준점임
- ✓ 이를 반복하는 Centroid Decomposition을 배워 보자

## Centroid란?

- ✓ 정점이  $N$  개인 트리에서 정점  $c$ 를 하나 제거하자
- ✓  $c$ 에 연결된 간선도 함께 제거함
- ✓ 남은 모든 연결 요소의 크기가  $N/2$  이하이면  $c$ 를 centroid라고 함
- ✓ 한국어로는 트리의 무게중심이라고 부름
- ✓ 여기서 크기는 연결 요소에 들어 있는 정점의 수임
- ✓ 정점이 하나뿐인 트리에서는 그 정점이 centroid임

$$\max_{\text{component } T_i \text{ of } T-c} |T_i| \leq \left\lfloor \frac{N}{2} \right\rfloor$$

## Centroid의 예시



- ✓ 정점 4를 제거하면  $\{1, 2, 3\}$  과  $\{5, 6, 7\}$  로 나뉨
- ✓ 두 연결 요소의 크기가 모두  $7/2$  이하이므로 4는 centroid임
- ✓ 정점 3을 제거하면 크기 4인 연결 요소가 남음
- ✓ 따라서 정점 3은 centroid가 아님

## Center와 Centroid

- ✓ Center는 다른 정점까지의 최대 거리를 최소화하는 정점임

$$\min_u \max_v \text{dist}(u, v)$$

- ✓ Centroid는 제거한 뒤 남는 연결 요소의 크기를 기준으로 함
- ✓ 두 개념이 항상 같은 정점을 선택하는 것은 아님

## 예시

- ✓ 경로  $a - b - c - d$ 에서  $a$ 에 앞 5개를 추가하자
- ✓ 전체 9개 정점 중 centroid는  $a$ , center는  $b$ 임

## Subtree Size

- ✓ 트리에 임의의 루트를 정하고 DFS를 수행하자
- ✓  $sz[u]$ :  $u$ 를 루트로 하는 서브트리의 정점 수

$$sz[u] = 1 + \sum_{v: \text{child of } u} sz[v]$$

- ✓ 정점  $u$ 를 제거했을 때 남는 연결 요소는 다음과 같음
  - 자식  $v$  방향: 크기  $sz[v]$
  - 부모 방향: 크기  $N - sz[u]$
- ✓ 루트의 부모 방향 크기는 0으로 처리함
- ✓ 부모 방향을 빠뜨리지 않는 것이 중요함

## DFS로 Centroid 찾기

- ✓ 정점  $u$  를 제거한 뒤 가장 큰 연결 요소의 크기를  $mx[u]$  라고 하자

$$mx[u] = \max \left( N - sz[u], \max_{v: \text{child of } u} sz[v] \right)$$

- ✓ 자식이 없는 경우 자식 방향의 최댓값은 0으로 둠
- ✓  $sz$  를 구한 뒤 모든 정점의  $mx$  를 확인함
- ✓ 다음 조건을 만족하는 정점이 centroid임

$$mx[u] \leq \left\lfloor \frac{N}{2} \right\rfloor$$

- ✓ 모든 centroid를 찾고 싶다면 조건을 만족하는 정점을 모두 저장함

## 큰 서브트리로 내려가기

- ✓  $sz$ 를 계산할 때 사용한 루트에서 시작하자
- ✓ 현재 정점의 자식 중  $sz[v] > N/2$ 인 자식이 있으면 그쪽으로 이동함
- ✓ 그런 자식은 동시에 둘 이상 존재할 수 없음
- ✓ 이동한 뒤 부모 방향의 크기는  $N - sz[v] < N/2$ 임
- ✓ 따라서 이후에는 자식 방향만 확인해도 됨
- ✓ 크기가  $N/2$ 를 넘는 자식이 없으면 현재 정점이 centroid임
- ✓ 매 이동에서 기준  $N$ 은 원래 탐색 대상 트리의 크기로 고정함

## Centroid가 항상 존재하는 이유

- ✓ 어떤 정점에서 크기가  $N/2$ 를 넘는 연결 요소가 있다고 하자
- ✓ 그 연결 요소 방향의 이웃으로 이동함
- ✓ 방금 떠난 정점 방향은 크기가  $N/2$ 보다 작아짐
- ✓ 따라서 다음 이동에서 같은 간선을 거슬러 돌아가지 않음
- ✓ 트리에는 사이클이 없으므로 같은 정점을 다시 방문할 수 없음
- ✓ 유한 번 이동한 뒤 더 이상 이동할 수 없는 정점에 도착함
- ✓ 이 정점에서는 모든 연결 요소의 크기가  $N/2$  이하임

## Centroid는 몇 개일까?

- ✓ 앞에서 트리에 centroid가 적어도 하나 존재함을 보였음
- ✓ 실제로 centroid의 개수는 항상 하나 또는 두 개임
- ✓ 두 개라면 서로 인접하며, 둘 사이 간선을 끊으면 크기가 같은 두 트리로 나뉨
- ✓ 따라서  $N$ 이 홀수이면 centroid는 정확히 하나임
- ✓  $N$ 이 짝수라고 해서 반드시 두 개인 것은 아님
- ✓ 이제 centroid 하나를 고정하고, 다른 centroid가 어디에 있을 수 있는지 보자

### 개수의 증명: 절반보다 작은 가지

- ✓ Centroid 하나를  $c$ 라고 하고,  $c$ 를 제거한 연결 요소를 가지라고 하자
- ✓ 크기가  $s < N/2$ 인 가지  $T$  안에서 정점  $x$ 를 하나 고르자
- ✓  $x$ 를 제거해도  $T$  바깥의 정점들은  $c$ 를 통해 모두 연결되어 있음
- ✓ 따라서  $c$ 가 속한 연결 요소의 크기는 적어도 다음과 같음

$$N - s > \frac{N}{2}$$

- ✓ 그러므로 이 가지 안에는 다른 centroid가 존재할 수 없음
- ✓ 모든 가지의 크기가  $N/2$ 보다 작다면  $c$ 가 유일한 centroid임
- ✓  $N = 1$ 인 경우에도 정점 하나만 있으므로 결론은 같음

## 개수의 증명: 정확히 절반인 가지

- ✓ 크기가  $N/2$ 인 가지  $T$ 는 최대 하나임
  - 두 개라면  $c$ 까지 합쳐 정점 수가  $N + 1$  이상이 됨
- ✓  $T$ 에서  $c$ 와 인접한 정점을  $v$ 라고 하자
- ✓  $v$ 를 제거하면  $c$  방향은  $N/2$ 개, 나머지 조각은 각각  $N/2 - 1$ 개 이하임
- ✓ 따라서  $v$ 도 centroid임
- ✓  $T$  안의 다른 정점  $x \neq v$ 를 제거하면  $c$  방향에는  $T$  바깥의  $N/2$ 개 정점과  $v$ 가 모두 남음
- ✓ 이 크기는  $N/2$ 를 넘으므로  $x$ 는 centroid가 아님
- ✓ 다른 가지도 앞의 증명으로 제외되므로 centroid는 정확히  $c, v$  두 개임

## Centroid 탐색의 시간 복잡도

- ✓ DFS로 모든  $sz$ 를 계산하는 데  $\mathcal{O}(N)$ 이 필요함
- ✓ 각 정점의 자식들을 확인하는 총횟수도  $\mathcal{O}(N)$ 임
- ✓ 따라서 centroid 하나 또는 전체를 찾는 시간은

$$\mathcal{O}(N)$$

- ✓ 인접 리스트와  $sz$  배열을 포함한 공간 복잡도는  $\mathcal{O}(N)$ 임
- ✓ 트리가 일자 모양이면 일반 DFS의 깊이는  $N$ 까지 커질 수 있음

## 트리에서 분할 정복하기

- ✓ 정점 하나를 기준으로 문제를 처리하고, 그 정점을 제거하자
- ✓ 남은 연결 요소마다 같은 문제를 재귀적으로 해결할 수 있음
- ✓ 하지만 아무 정점이나 고르면 분할이 한쪽으로 치우칠 수 있음
- ✓ 일자 트리의 끝점만 제거하면 크기가  $N, N - 1, N - 2, \dots$ 가 됨

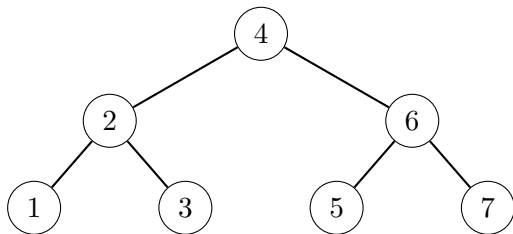
$$N + (N - 1) + \dots + 1 = \mathcal{O}(N^2)$$

- ✓ 매번 centroid를 선택하면 각 연결 요소의 크기가 절반 이하가 됨

## Centroid Decomposition

- ✓ 현재 연결 요소의 크기를  $S$  라고 하자
- ✓ 다음 과정을 재귀적으로 반복함
  1. 현재 연결 요소의 centroid  $c$  를 찾음
  2. 현재 단계에서  $c$  를 기준으로 필요한 정보를 처리함
  3.  $c$  를 제거한 것으로 표시함
  4. 남은 각 연결 요소에 대해 재귀 호출함
- ✓ 각 재귀 호출의 centroid 를 부모  $c$  에 연결하면 새로운 트리가 됨
- ✓ 이 트리를 centroid tree 라고 부름
- ✓ 센트로이드 트리의 간선은 원래 트리에 없는 간선일 수도 있음

## Centroid Tree의 예시



- ✓ 원래 트리는 경로  $1 - 2 - 3 - 4 - 5 - 6 - 7$ 임
- ✓ 4를 먼저 선택한 뒤, 왼쪽에서는 2, 오른쪽에서는 6을 선택함
- ✓ 마지막으로 정점 1, 3, 5, 7을 각각 선택함

## Centroid Tree의 높이

- ✓ 한 정점이 속한 연결 요소의 크기를 따라가 보자
- ✓ 재귀 단계가 내려갈 때마다 크기가 절반 이하로 줄어듦

$$N, \quad \frac{N}{2}, \quad \frac{N}{4}, \quad \dots$$

- ✓ 따라서 센트로이드 트리의 높이는  $\mathcal{O}(\log N)$  임
- ✓ 각 정점은 자신을 포함해  $\mathcal{O}(\log N)$  개의 센트로이드 조상을 가짐
- ✓ 원래 트리의 높이가  $N$  이어도 이 성질은 성립함

## Centroid Decomposition의 복잡도

- ✓ 크기  $S$ 인 연결 요소에서 centroid를 찾는 비용은  $\mathcal{O}(S)$ 임
- ✓ 같은 재귀 깊이에 있는 연결 요소들은 서로 정점을 공유하지 않음
- ✓ 따라서 한 깊이에서 처리하는 정점 수의 합은 최대  $N$ 임
- ✓ 깊이가  $\mathcal{O}(\log N)$ 이므로 분할 구조를 만드는 시간은

$$\mathcal{O}(N \log N)$$

- ✓ 각 단계의 추가 작업도  $\mathcal{O}(S)$ 이면 전체 복잡도는 같음
- ✓ 추가 작업에 정렬이나 트리 자료구조를 쓰면 로그 인자가 더 붙을 수 있음

## 제거 표시와 현재 연결 요소

- ✓ 실제 인접 리스트에서 정점이나 간선을 지울 필요는 없음
- ✓  $blocked[u]$  로 이미 선택한 centroid 인지 표시함
- ✓ 크기 계산과 거리 수집 DFS에서는 제거된 정점을 건너뛴
- ✓ 매 재귀 호출에서 현재 연결 요소의  $sz$  를 다시 계산함
- ✓ centroid 판정에는 전체  $N$  이 아닌 현재 크기  $S$  를 사용함
- ✓ 현재 단계의 처리를 마친 뒤  $c$  를 표시하고 남은 이웃으로 재귀함
- ✓  $c$  를 먼저 표시한다면 거리 수집은 각 이웃에서 따로 시작해야 함

## 경로를 두 종류로 나누기

- ✓ 현재 연결 요소의 centroid를  $c$ 라고 하자
- ✓ 두 정점  $u, v$ 를 잇는 경로는 다음 중 하나임
  - $c$ 를 지나는 경로
  - $c$ 를 제거한 뒤 하나의 연결 요소 안에 남는 경로
- ✓ 첫 번째 종류는 현재 단계에서 처리함
- ✓ 두 번째 종류는 해당 연결 요소의 재귀 호출에서 처리함
- ✓ 서로 다른 연결 요소의 정점들을 잇는 경로는 반드시  $c$ 를 지남
- ✓ 끝점 중 하나가  $c$ 인 경로도 첫 번째 종류에 포함함

## 각 경로를 정확히 한 번 처리하기

- ✓ 두 끝점이 같은 연결 요소에 남으면 그 연결 요소 안으로 내려감
- ✓ 처음으로 서로 다른 조각에 들어가거나 한 끝점이 centroid가 되는 순간이 있음
- ✓ 그 단계의 centroid를  $c$ 라고 하면 경로가  $c$ 를 지남
- ✓ 이 경로는 현재 단계에서 한 번 처리하고 더 아래에서는 처리하지 않음
- ✓  $c$ 는 센트로이드 트리에서 두 끝점의 LCA에 해당함
- ✓ 여기서 LCA는 원래 트리에서 구한 LCA와 다를 수 있음

$$\text{dist}(u, v) = \text{dist}(u, c) + \text{dist}(c, v)$$

### 기초 예제: 길이가 $K$ 인 경로

- ✓ 간선의 길이가 모두 1인 트리에서 길이가  $K \geq 1$ 인 경로를 세자
- ✓ 두 끝점의 순서는 구분하지 않음



- ✓  $K = 2$ 이면  $(1, 3), (2, 4), (3, 5)$ 의 세 경로가 있음
- ✓ Centroid 3을 기준으로 양쪽 가지의 거리 목록은 모두  $[1, 2]$ 임
- ✓ 먼저 centroid를 지나는 경로를 세고, 가지 내부는 재귀로 처리함

## 기초 예제: 조회한 뒤 합치기

- ✓  $cnt[d]$ : 이전에 처리한 가지에서 centroid까지의 거리가  $d$ 인 정점 수
- ✓ Centroid 자신을 포함하도록  $cnt[0] = 1$ , 나머지는 0으로 시작함
- ✓ 현재 가지의 거리 목록  $D$ 를 DFS로 구한 뒤 다음 순서로 처리함
  1. 모든  $d \in D$ 에 대해  $cnt[K - d]$ 를 답에 더함
  2. 모든 조회가 끝난 뒤  $cnt[d]$ 를 1씩 증가시킴
- ✓ 거리  $d > K$ 인 정점은 건너뛴
- ✓ 조회와 삽입을 섞으면 같은 가지의 정점끼리도 짝지어지므로 주의함
- ✓ 모든 가지를 처리한 뒤 각 가지 내부에서 같은 과정을 반복함

### 기초 예제: 직접 계산하기

- ✓ 앞의 일자 트리에서  $K = 2$ , centroid는 3임
- ✓ 왼쪽 가지의 거리 목록  $[1, 2]$ 를 조회하면

$$cnt[1] + cnt[0] = 0 + 1 = 1$$

- ✓ 왼쪽 가지를 합치면  $cnt[0] = cnt[1] = cnt[2] = 1$ 이 됨
- ✓ 오른쪽 가지의 거리 목록  $[1, 2]$ 를 조회하면

$$cnt[1] + cnt[0] = 1 + 1 = 2$$

- ✓ 두 가지 내부에는 길이 2인 경로가 없으므로 최종 답은  $1 + 2 = 3$ 임

## 기초 예제: 시간 복잡도와 구현

- ✓ 크기  $S$ 인 연결 요소에서 거리 수집과 배열 처리는  $\mathcal{O}(S)$  임
- ✓ 센트로이드 분할의 각 단계에서 선형 시간만 사용하므로 전체 시간은

$$\mathcal{O}(N \log N)$$

- ✓  $cnt$ 는 현재 단계가 끝나면 값을 바꾼 인덱스만 초기화함
- ✓ 매 centroid마다 길이  $N$  전체를 초기화하면  $\mathcal{O}(N^2)$ 이 될 수 있음